Strategic Implementation Blueprint: The Agentic Android DevOps Lifecycle using Claude Code CLI

1. The Paradigm Shift in Android Engineering: From IDE-Centric to Agent-Centric Development

The landscape of mobile application development, specifically within the Android ecosystem, stands at a critical inflection point. For over a decade, the primary interface for engineering has been the Integrated Development Environment (IDE), predominantly Android Studio. While effective for code syntax and compilation, the IDE model inherently fragments the developer's cognitive load across disjointed utilities: the Gradle build system for dependency management, the Android Debug Bridge (ADB) for device orchestration, Git for version control, and web browsers for documentation or ticketing systems.[1] This fragmentation creates significant context switching costs, which lowers productivity and increases the likelihood of human error during complex operations like release management or refactoring legacy codebases.

The introduction of the **Claude Code CLI** represents a fundamental architectural shift toward an **Agentic Developer Experience (ADX)**. Unlike traditional autocomplete assistants that live passively within the editor, Claude Code operates as an autonomous agent in the terminal, capable of orchestrating tools, reasoning about system state, and executing complex, multi-step workflows.[3] For a Senior Android DevOps team, this is not merely a productivity booster; it is an infrastructure layer that allows us to codify our architectural patterns—Clean Architecture, MVVM, and Material Design 3—into active agents that enforce standards and execute toil autonomously.[5]

1.1 The Architecture of Agency in Android

To successfully adopt this technology, we must conceptualize the environment as a three-tiered architecture. At the base layer lies the **Physical Infrastructure**, comprising the local development machine, connected Android devices (emulators or physical hardware), and the remote CI/CD pipelines (e.g., GitHub Actions, Bitrise). The middle layer is the **Model Context Protocol (MCP) Fabric**, a universal connectivity standard that acts as the nervous system, translating the raw inputs from Gradle daemons and ADB streams into structured context that the Large Language Model (LLM) can comprehend.[7] The top layer is the **Agentic Orchestration Layer**, consisting of Claude Code CLI, specialized Subagents, and Slash Commands that serve as the interface for developer intent.[9]

This report outlines the strategic implementation of this architecture. We will move beyond simple prompt engineering to define a rigorous engineering environment where "Skills" are executable binaries, "Subagents" are domain experts with distinct context windows, and "Slash Commands" are the codified workflows of our most senior engineers.

---

2. Infrastructure & Environment Configuration: The Agent's Foundation

The efficacy of an AI agent is directly proportional to the quality and relevance of the context it can access. In an Android project, providing the entire codebase to the context window is detrimental due to "context bloat," which degrades reasoning performance and increases costs.[10] Therefore, the foundational configuration must focus on **Context Curation** and **Permission Governance**.

2.1 Global Context Memory: The CLAUDE.md Strategy

The CLAUDE.md file serves as the long-term memory for the agent, loaded at the start of every session.[4] For an Android engineering team, this file must function as the "Architectural North Star." It is insufficient to simply list commands; we must explicitly define the negative constraints—what the agent must *not* do—to prevent the generation of deprecated code patterns that plague the Android ecosystem (e.g., AsyncTask, findViewById, or Synthetics).

The following configuration strategy for CLAUDE.md ensures that every interaction adheres to our specific tech stack:

Table 1: CLAUDE.md Configuration Components for Android

Component	Configuration Strategy	Rationale
Build System	Explicitly define the use of Version Catalogs (libs.versions.toml) and Kotlin DSL (build.gradle.kts).	Prevents the agent from suggesting Groovy syntax or hardcoding dependencies, which breaks build caching.[12]
Architecture	Mandate Clean Architecture: Domain (Pure Kotlin) -> Data (Repositories) -> UI (MVVM	Enforces separation of concerns and prevents domain logic leakage into ViewModels.[5]

	+ Compose).	
UI Framework	Enforce Material 3 and Jetpack Compose. Explicitly ban XML layouts for new features.	Ensures compliance with modern Android UI standards and prevents regression to legacy View systems.[14]
Concurrency	Mandate Coroutines and Flow. Ban RxJava for new implementations and Thread instantiation.	Aligns with the structured concurrency model required for lifecycle-aware operations.[6]
Testing	Define stack: JUnit 5, MockK, Turbine. Require tests for every logical change.	Prevents the agent from generating JUnit 4 style tests or missing coroutine test dispatchers.[16]

Implementation Detail:
The CLAUDE.md must be placed in the project root. Furthermore, to manage token usage effectively, we must implement a rigorous .claudeignore file. Android builds generate massive artifacts (APKs, AARs, R.java files) that contain no semantic value for the agent but consume massive context.10
File: .claudeignore

Build Artifacts

build/
app/build/
.gradle/
**/build/

IDE Metadata

.idea/
*.iml
local.properties

Assets impacting token limits

*.svg
*.png

*.hprof
*.aprof
By explicitly excluding *.hprof (heap dumps) and build/ directories, we prevent the agent from attempting to read binary blobs, ensuring it focuses solely on source code and relevant configuration files.4

2.2 Permission Governance and Security

Claude Code operates with extensive privileges on the local machine. In a DevOps capacity, where the agent might execute git push or adb uninstall, unbridled access presents a risk. We must utilize the settings.json configuration to establish a "Least Privilege" model.[17]

The governance model differentiates between **Safe Tools** (read-only, analysis) and **Sensitive Tools** (write, delete, deploy). Safe tools are added to the allow list to reduce friction—developers should not have to approve every ls or grep command. Sensitive tools remain protected, requiring explicit human confirmation.

Configuration: managed-settings.json (Corporate Policy)
We deploy a managed settings file to enforce security policies across the entire engineering team, ensuring that individual developers cannot override critical safety checks.17

JSON

```
{
  "permissions": {
    "allow":,
    "deny":
  },
  "autoUpdaterStatus": "disabled"
}
```

This configuration ensures that while the agent can freely explore the codebase and device state, it cannot destructively remove data or expose signing keys without the developer's direct intervention.[17]

---

3. The Model Context Protocol (MCP) Architecture

The core differentiator of this implementation plan is the aggressive use of the **Model Context Protocol (MCP)**. Without MCP, Claude Code interacts with the Android project as a collection of text files. With MCP, it interacts with the *system*—understanding the build graph, the running device state, and the git history as structured data.[7] We will deploy three critical

MCP servers: Gradle, Android ADB, and a custom Architecture Inspector.

3.1 The Gradle MCP Server: Programmatic Build Intelligence

The Gradle build system is notoriously complex, utilizing a daemon process and a configuration cache that opaque text files (build.gradle.kts) do not fully reveal. A simple text read of a build file does not tell the agent about resolved dependency versions or task execution graphs. We will implement the gradle-mcp-server to bridge this gap.[16]

Operational Mechanism:
The Gradle MCP server connects to the Gradle Tooling API. This allows Claude to query the project model directly. When a developer asks, "Why is the build failing?", instead of blindly guessing based on error logs, the agent can invoke get_gradle_project_info or execute_gradle_task to retrieve a structured diagnostic report.16
Implementation Configuration (.claude/mcp.json):

JSON

```
{
  "mcpServers": {
    "gradle": {
      "command": "java",
      "args": [
        "-jar",
        "${HOME}/.mcp/gradle-mcp-server-all.jar",
        "--stdio"
      ],
      "env": {
        "JAVA_HOME": "${HOME}/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home"
      }
    }
  }
}
```

Strategic Value:

- **Dependency Resolution:** The agent can accurately identify version conflicts (e.g., androidx.core:core-ktx version mismatch) by querying the resolved dependency graph rather than parsing the declaration file.[13]
- **Test Execution:** The run_gradle_tests tool allows the agent to execute specific test suites and receive hierarchical JSON output of failures, enabling it to pinpoint exactly

which assertion failed without parsing massive HTML reports.[16]

3.2 The Android ADB MCP Server: Device Agency

To function as a true Android DevOps expert, the agent must "see" and "touch" the device. We will integrate an ADB MCP server (e.g., mobile-mcp or a custom implementation) that wraps the Android Debug Bridge commands into callable tools.[1]

Capabilities Integration:

1. **Logcat Filtering:** Instead of dumping the full log stream, the agent uses an MCP tool get_logcat with parameters for tag, priority, and pid. This is critical for token optimization, allowing the agent to request "only logs from com.myapp with level ERROR from the last 5 minutes".[23]
2. **UI Inspection:** Using take_screenshot and get_accessibility_node_tree, the agent can correlate the visual state of the app with the codebase. For example, if a UI test fails, the agent can screenshot the emulator, analyze the accessibility tree to see what elements are visible, and diagnose why a ComposeTestRule failed to find a node.[25]

Configuration Strategy:
We configure the ADB MCP server to strictly scope its operations to the package name defined in the project, preventing the agent from interacting with other apps on the developer's device.

3.3 Custom "Architecture Scout" MCP

To support our specific Clean Architecture requirements, we will implement a lightweight, custom MCP server using the Node.js SDK. This server, architecture-scout, acts as a linter-on-steroids.

Functionality:

- **Layer Verification:** It scans package structures to ensure that Domain layer classes do not import android.* packages (a strict violation of Clean Architecture).[6]
- **Naming Conventions:** It enforces that Repositories end in Repository and UseCases in UseCase, rejecting deviations programmatically before they reach code review.

This moves architectural governance from "reactive code review" to "proactive agentic checks".[27]

---

4. The Subagent Ecosystem: Specialized Expert Personas

A single, general-purpose agent often struggles to maintain the specific nuances of different

domains (e.g., UI design vs. Build engineering) within a long conversation. To solve this, we will deploy a **Subagent Ecosystem**. Subagents are specialized instances of Claude, each with a dedicated context window, specific tool access, and a hyper-focused system prompt.[9]

This separation of concerns mirrors a high-performing engineering team: you have an Architect, a QA Engineer, a Release Manager, and a UI Designer. We will instantiate these roles virtually.

4.1 Subagent 1: The AndroidArchitect (Code Owner)

The AndroidArchitect is the guardian of code quality and pattern adherence. It is the only agent authorized to perform large-scale refactoring.

Definition File: .claude/agents/architect.md

YAML

```yaml
name: android-architect
description: Principal Android Engineer specializing in Clean Architecture, MVVM, and Jetpack Compose. Use strictly for refactoring, feature implementation, and architectural review.
tools:
model: claude-3-5-sonnet-20241022
```

System Prompt Design:
The system prompt for this subagent is rigorously crafted to enforce the "Senior" persona. It explicitly overrides generic coding behaviors with our specific stack requirements:

> "You are a Principal Android Engineer. Your code must be production-ready, immutable, and thread-safe.
> Architecture: Adhere strictly to MVVM + Clean Architecture.
> Domain Layer: Pure Kotlin. No Android dependencies. Use suspend operator fun invoke for UseCases.
> Data Layer: Use Repository pattern. Map all data to Domain Entities. Use runCatching for error handling.
> UI Layer: Jetpack Compose only. No Fragments. State must be hoisted. ViewModels must expose a single StateFlow<UiState>.
> Prohibitions: Never use AsyncTask, LiveData (unless migration), or Manual DI (Use Hilt)."

Use Case: A developer asks, "Refactor this Activity to MVVM." The AndroidArchitect subagent activates, reads the legacy code, creates the ViewModel, UseCase, and Repository interfaces, and rewrites the UI in Compose, all while maintaining a separate context window so the main

chat remains clear.[9]

4.2 Subagent 2: The GradleGuru (Build Engineer)

Gradle builds are the most common source of frustration in Android development. The GradleGuru subagent is optimized for build introspection and dependency management.

Definition File: .claude/agents/gradle-guru.md

YAML

```yaml
name: gradle-guru
description: Expert Build Engineer. Specialized in Gradle Kotlin DSL, Version Catalogs, and CI/CD pipelines. Use for fixing build errors, updating dependencies, and optimizing build speed.
tools:
model: claude-3-5-sonnet-20241022
```

System Prompt Design:

> "You are a Gradle Build Expert. You prioritize build speed and reproducibility.
> Dependency Management: Always use the Version Catalog (libs.versions.toml). Never hardcode versions in build.gradle.kts.
> Debugging: When a build fails, use the gradle MCP tool to inspect the task graph. Analyze the stacktrace to identify the root cause (e.g., manifest merge errors, KAPT failures).
> Optimization: Suggest disabling Jetifier if not needed. Ensure kapt is replaced by ksp where possible."

Strategic Insight: By giving this agent the MCP(gradle) tool, it can actively run ./gradlew dependencies or query the tooling model to resolve version conflicts that a human would struggle to visualize.[13]

4.3 Subagent 3: The DroidQA (Test Automation Engineer)

This agent focuses on the "Testing Pyramid." It is responsible for writing and running tests, ensuring that new features do not introduce regressions.

Definition File: .claude/agents/droid-qa.md

YAML

```yaml
name: droid-qa
description: QA Automation Engineer. Expert in JUnit 5, MockK, Espresso, and Compose Testing. Use for running tests, analyzing failures, and generating test cases.
tools:
model: claude-3-haiku-20240307
```

System Prompt Design:

> "You are a QA Automation Engineer. You value high code coverage and stable tests.
> Unit Tests: Use MockK for mocking. Use turbine for testing Kotlin Flows.
> UI Tests: Use ComposeTestRule. Ensure content descriptions are present for accessibility.
> Failure Analysis: When analyzing a failure, read the full stack trace, identify the assertion error, and cross-reference with the source code."

Efficiency Note: We utilize the Haiku model for this agent because test log analysis involves processing large volumes of text (logs) where reasoning depth is less critical than speed and context window size.[28]

4.4 Subagent 4: The ReleaseOps (Deployment Manager)

This agent manages the release lifecycle, interacting with Git tags, Changelogs, and Fastlane.

Definition File: .claude/agents/release-ops.md

YAML

```yaml
name: release-ops
description: DevOps specialist for Android releases. Handles versioning, changelogs, git tagging, and Fastlane execution.
tools:
```

System Prompt Design:

> "You manage the release pipeline.
> Versioning: Follow Semantic Versioning. Update versionCode and versionName in libs.versions.toml.

Changelog: Parse git history between tags to generate user-facing changelogs.
Fastlane: Validation of Fastfile configurations before execution."

---

5. The Slash Command Library: Codifying Senior Workflows

Slash Commands allow us to package complex, multi-step prompting strategies into reusable shortcuts. This effectively democratizes the knowledge of senior engineers; a junior developer can execute a "Senior-level" refactor simply by invoking the correct command.[29] We will implement a comprehensive library of commands stored in .claude/commands/.

5.1 Development Workflow Commands

Command: /feature-scaffold

- **Purpose:** Generates the boilerplate for a new feature following Clean Architecture.
- **Implementation (.claude/commands/feature-scaffold.md):**

description: Scaffold a new Clean Architecture feature module. Usage: /feature-scaffold [FeatureName] allowed-tools:

Task: Scaffold Feature "$1"

1. **Create Directory Structure**:
   - feature/$1/src/main/java/com/company/feature/$1/domain (UseCase, Repository)
   - feature/$1/src/main/java/com/company/feature/$1/data (RepositoryImpl, DataSource)
   - feature/$1/src/main/java/com/company/feature/$1/presentation (ViewModel, Screen)
2. **Gradle Setup**:
   - Create feature/$1/build.gradle.kts applying android-library, kotlin-android, and hilt.
   - Add module to settings.gradle.kts.
3. **Architecture Components**:
   - Generate Feature$1Navigation interface.

    - Generate Feature$1ViewModel extending BaseViewModel.
      Insight: This command eliminates the 30-60 minutes of manual setup required to create a new module, wire it up in Gradle, and create the package structure, ensuring consistency across all features.6

Command: /refactor-compose

- **Purpose:** Converts a legacy XML layout file to a Jetpack Compose screen.
- Implementation:
  Uses the AndroidArchitect subagent. It takes an XML file path as an argument, reads it, analyzes the View hierarchy (ConstraintLayout, TextViews), and maps them to Compose equivalents (Column, Text), preserving attributes like padding and logic.32

5.2 Debugging & Diagnostics Commands

Command: /analyze-anr

- **Purpose:** Diagnoses Application Not Responding (ANR) errors.
- **Implementation (.claude/commands/analyze-anr.md): description: Analyze ANR traces from the connected device. allowed-tools: Task: ANR Analysis**
  1. **Fetch Traces**: Run adb pull /data/anr/traces.txt./temp_anr_trace.txt.
  2. **Identify Blockage**: Search the trace for "main" thread state. Look for "Held by" or "Waiting to lock".
  3. **Correlate**: Run adb logcat -d -t 1000 to see what happened immediately before the freeze.
  4. Report: Summarize the deadlock or main-thread IO operation causing the ANR.
     Insight: ANR debugging typically involves cryptic trace files. This command automates the retrieval and parsing, presenting the developer with a clear "Smoking Gun" analysis.34

Command: /crash-report

- **Purpose:** Instant crash analysis.
- Implementation:
  Executes adb logcat -b crash -d. If empty, scans the main buffer for fatal exceptions. It parses the stack trace, identifies the line of code in the local project, reads that file, and suggests a fix.35

5.3 DevOps & Release Commands

Command: /bump-version

- **Purpose:** Automates versioning pre-release.
- **Implementation (.claude/commands/bump-version.md):**
  description: Increment version code and update version name. Usage: /bump-version [major|minor|patch] argument-hint: [major|minor|patch] allowed-tools:
  Task: Version Bump
  1. **Read Config**: Open libs.versions.toml.
  2. **Parse**: Extract versionCode (int) and versionName (semver string).
  3. **Increment**:
     - Increase versionCode by 1.
     - Logic for versionName:
       - If arg is 'patch', 1.2.3 -> 1.2.4.
       - If arg is 'minor', 1.2.3 -> 1.3.0.
  4. Commit: Git commit with message "chore: prepare release x.y.z".
     Insight: This command utilizes the Read and Edit tools to manipulate the TOML file safely, preventing syntax errors that could break the build.36

Command: /ci-local

- **Purpose:** "Shift-Left" testing. Simulates the CI pipeline locally.
- Implementation:
  Runs a sequence: ./gradlew lintDebug, followed by ./gradlew testDebugUnitTest. If these pass, it provides a "Ready to Push" certification. If they fail, it invokes the DroidQA subagent to explain the failure.

---

6. Implementation of Virtual Skills

While Claude Code consolidates "Skills" into MCP tools and Subagents, specific atomic capabilities often require custom scripting. We define these as "Virtual Skills"—scripts that are exposed to Claude via allowed-tools or MCP, which perform complex logic that an LLM cannot do reliably with simple text generation.

6.1 Skill: Visual Regression Check (pixel-perfect)

To verify UI fidelity, we cannot rely on the LLM's "imagination." We implement a Python script using OpenCV and ADB.

- Script (scripts/visual_check.py):
  Takes a screenshot of the device (adb shell screencap), pulls it, and compares it against a reference image in the repository. It outputs a diff percentage and a generated image highlighting differences.
- Integration:
  This script is wrapped in a Slash Command /visual-check. Claude executes the script via Bash, reads the output (diff percentage), and if the diff is > 0%, it reports the failure to the developer.

6.2 Skill: Unused Resource Cleaner (res-cleaner)

Android projects accumulate unused resources (strings, drawables) over time.

- Mechanism:
  We expose the lint tool's specific "UnusedResources" check.
  Command: ./gradlew lintDebug -Pcheck=UnusedResources.
  Claude parses the XML report, identifies the file paths of unused resources, and offers to delete them. This turns a passive report into an active maintenance task.

---

7. Change Management and Success Metrics

Implementing this agentic workflow requires a cultural shift. Senior developers may be skeptical of "AI writing code." The rollout must focus on **Augmentation**, not replacement.

7.1 Phased Rollout Plan

1. **Phase 1: The Observer (Week 1-2):** Deploy gradle-mcp and logcat tools. Developers use Claude Code primarily for debugging and log analysis. *Goal:* Trust building.
2. **Phase 2: The Assistant (Week 3-4):** Deploy feature-scaffold and bump-version commands. Developers use agents for boilerplate generation and toil reduction. *Goal:* Efficiency.
3. **Phase 3: The Partner (Month 2+):** Deploy AndroidArchitect for active refactoring and code reviews. *Goal:* Quality enforcement.

7.2 Metrics for Success

To evaluate the ROI of this implementation, we will track specific engineering metrics:

Table 2: Key Performance Indicators (KPIs)

Metric	Measurement Method	Target Outcome
Debug Latency	Time from "Build Failed" to "Build Successful".	**30% Reduction.** (Agents diagnose Gradle errors faster than humans searching StackOverflow).
Context Switching	Number of times a developer switches focus away from the terminal/IDE.	**50% Reduction.** (Logs, Git, and Build happen inside the Claude session).
Boilerplate Time	Time to create a new feature module skeleton.	**90% Reduction.** (From ~30 mins manual to ~2 mins via /feature-scaffold).
CI Stability	Frequency of build failures on the main branch.	**20% Improvement.** (Local /ci-local checks catch errors before push).

7.3 Risk Mitigation: The "Hallucination" Trap

A primary risk is the agent generating code that compiles but is incorrect (e.g., swallowing exceptions).

- **Mitigation:** The AndroidArchitect subagent's system prompt strictly enforces runCatching and explicit error handling.
- **Mitigation:** The DroidQA agent must be invoked to write a test for every piece of code generated by the Architect. The rule "No Code Without Tests" is enforced by the Slash Command workflow.

8. Conclusion

By implementing this comprehensive plan, the Android engineering organization transitions from a tool-fragmented workflow to a cohesive **Agentic DevOps Environment**. The integration of **Gradle and ADB MCP servers** provides the agent with the necessary ground truth to act reliably. The **Subagent Ecosystem** ensures that context is isolated and expertise

is applied correctly, whether in architecture, build engineering, or QA. Finally, the **Slash Command Library** codifies the tacit knowledge of senior engineers into executable assets. This architecture does not just automate tasks; it elevates the baseline quality of the entire codebase by embedding architectural standards directly into the tools developers use every day.

Works cited

1. Bridging Android Development and Language Models with MCP, accessed January 9, 2026, https://pub.towardsai.net/bridging-android-development-and-language-models-with-mcp-0cb90335ffbf
2. How do I use Claude Code for DevOps tasks? - Milvus, accessed January 9, 2026, https://milvus.io/ai-quick-reference/how-do-i-use-claude-code-for-devops-tasks
3. Code execution with MCP: building more efficient AI agents - Anthropic, accessed January 9, 2026, https://www.anthropic.com/engineering/code-execution-with-mcp
4. Claude Code: Best practices for agentic coding - Anthropic, accessed January 9, 2026, https://www.anthropic.com/engineering/claude-code-best-practices
5. A quest for comfy Android app architecture pt. 2— the Clean, accessed January 9, 2026, https://proandroiddev.com/a-quest-for-comfy-android-app-architecture-pt-2-the-clean-3a96eb8b8429
6. Build Better Android Apps with MVVM & Clean Architecture | iFlair, accessed January 9, 2026, https://www.iflair.com/building-better-android-apps-with-mvvm-and-clean-architecture/
7. Model Context Protocol, accessed January 9, 2026, https://modelcontextprotocol.io/
8. Introducing the Model Context Protocol - Anthropic, accessed January 9, 2026, https://www.anthropic.com/news/model-context-protocol
9. Subagents - Claude Code Docs, accessed January 9, 2026, https://code.claude.com/docs/en/sub-agents
10. LLM Context Management: How to Improve Performance and Lower ..., accessed January 9, 2026, https://eval.16x.engineer/blog/llm-context-management-guide
11. How to Use Claude Code: A Guide to Slash Commands, Agents ..., accessed January 9, 2026, https://www.producttalk.org/how-to-use-claude-code-features/
12. Configure a Gradle project | Kotlin Documentation, accessed January 9, 2026, https://kotlinlang.org/docs/gradle-configure-project.html
13. Dependencies versions in Gradle Kotlin DSL | by Kamil Seweryn, accessed January 9, 2026, https://proandroiddev.com/dependencies-versions-in-gradle-kotlin-dsl-a8db15cedee2
14. Material Design 3 in Compose - Android Developers, accessed January 9, 2026,

https://developer.android.com/develop/ui/compose/designsystems/material3
15. Material 3 Design System. Personal, Adaptive, and Expressive by..., accessed January 9, 2026, https://zoewave.medium.com/material-3-design-system-e91a15d303a0
16. IlyaGulya/gradle-mcp-server: A Model Context Protocol ... - GitHub, accessed January 9, 2026, https://github.com/IlyaGulya/gradle-mcp-server
17. Claude Code settings - Claude Code Docs, accessed January 9, 2026, https://code.claude.com/docs/en/settings
18. Gradle MCP Server - SourceForge, accessed January 9, 2026, https://sourceforge.net/projects/gradle-mcp-server.mirror/files/v0.1.0/gradle-mcp-server-all.jar/download
19. Gradle MCP Server by Ilya Gulya: Your AI Co-pilot for Build ..., accessed January 9, 2026, https://skywork.ai/skypage/en/gradle-mcp-server-ai-build-automation/1981648013068701696
20. Gradle MCP Server - LobeHub, accessed January 9, 2026, https://lobehub.com/pl/mcp/ilyagulya-gradle-mcp-server
21. Mobile MCP - iOS/Android automation and development - MCP Server, accessed January 9, 2026, https://cursor.directory/mcp/mobile-mcp-ios-android-automation-and-development
22. ADB MCP Server - playbooks, accessed January 9, 2026, https://playbooks.com/mcp/srmorete-adb
23. Jacksgong/okcat: The powerful log analysis, you can use it ... - GitHub, accessed January 9, 2026, https://github.com/Jacksgong/okcat
24. Filtering Android adb logcat efficiently in bash command line - Medium, accessed January 9, 2026, https://medium.com/@hissain.khan/filtering-android-adb-logcat-efficiently-in-bash-command-line-4992fb1acd61
25. Expose devices as an MCP service - Midscene - Joyful UI Automation, accessed January 9, 2026, https://midscenejs.com/mcp
26. An AI Engineer's Guide to Mastering Android with Natural Language, accessed January 9, 2026, https://skywork.ai/skypage/en/droidmind-ai-engineer-guide-android/1980551946369093632
27. Build-Scout | Awesome MCP Servers, accessed January 9, 2026, https://mcpservers.org/servers/David-Parry/build-scout
28. What's your best way to use Sub-agents in Claude Code so far?, accessed January 9, 2026, https://www.reddit.com/r/ClaudeAI/comments/1mdyc60/whats_your_best_way_to_use_subagents_in_claude/
29. Slash commands - Claude Code Docs, accessed January 9, 2026, https://code.claude.com/docs/en/slash-commands
30. How to Create Custom Slash Commands in Claude Code, accessed January 9, 2026, https://en.bioerrorlog.work/entry/claude-code-custom-slash-command

31. alexanderommel/Android-Modular-Clean-Architecture-MVVM - GitHub, accessed January 9, 2026, https://github.com/alexanderommel/Android-Modular-Clean-Architecture-MVVM
32. How I use Claude Code (+ my best tips) - Builder.io, accessed January 9, 2026, https://www.builder.io/blog/claude-code
33. Now in Android: a Material 3 case study | by Manda Edling - Medium, accessed January 9, 2026, https://medium.com/androiddevelopers/now-in-android-a-material-3-case-study-21e44bdfd2bc
34. Logcat command-line tool | Android Studio, accessed January 9, 2026, https://developer.android.com/tools/logcat
35. ADB logcat: Options, Filters & More - GitHub, accessed January 9, 2026, https://github.com/lana-20/adb-logcat-options-filters
36. Bump version code for Android apps using GitHub Actions, accessed January 9, 2026, https://dev.to/rockandnull/bump-version-code-for-android-apps-using-github-actions-436d?url=https://dev.to/rockandnull/bump-version-code-for-android-apps-using-github-actions-436d
37. Increase version code and version name on Android using Gradle ..., accessed January 9, 2026, https://medium.com/@IlyaEremin/npm-version-for-gradle-android-9137a7dc273c